

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Информационная безопасность

Работа №2

**«Анализ и устранение уязвимости на примере реального CVE с
использованием Vulhub»**

Болорболд Аригуун

Группа: P3411

Выполнение

Выбранная уязвимость

В рамке этой работы была выбрана уязвимость CVE-2016-10033, касающегося WordPress версии 4.6. Злоумышленник может воспользоваться этой уязвимостью для выполнения произвольного кода в контексте приложения. Неудачная попытка эксплойта приведет к отказу в обслуживании.

Последовательность действий по воспроизведению уязвимости

- 1. Клонировать репозиторию Vulhub, точнее папку с выбранной уязвимостью;
(<https://github.com/vulhub/vulhub/tree/master/wordpress/pwnscriptum>)
- 2. Изучение приложенного файла README.md в папке;
- 3. Модификация скриптов и файлов конфигурации (изначальные состояния не подходили под ОС Windows 11);
- 4. Запуск образа Docker через *docker-compose up -d*;

☐	▼	●	pwnscriptum	-	-	-	0.14%	⚙	■	⋮	🗑
☐		●	web-1	5efd0ff37a90	vulhub/wor	8080:80	0.01%	⚙	■	⋮	🗑
☐		●	mysql-1	98d116e2e441	mysql:5		0.1%	⚙	■	⋮	🗑
☐		●	payload-1	582b5978638f	pwnscriptu		0.03%	⚙	■	⋮	🗑

Рисунок 1 — Запущенные образы Docker для воспроизведения уязвимости

- 5. Доступ к адресу <http://127.0.0.1:8080/> и настройка WordPress;



Welcome

Welcome to the famous five-minute WordPress installation process! Just fill in the information below and you'll be on your way to using the most extendable and powerful personal publishing platform in the world.

Information needed

Please provide the following information. Don't worry, you can always change these settings later.

Site Title

IS#2

Username

admin

Usernames can have only alphanumeric characters, spaces, underscores, hyphens, periods, and the @ symbol.

Password

rGYq@iM0^@xlv#9L

Strong

Hide

Important: You will need this password to log in. Please store it in a secure location.

Your Email

bolorboldariguun16@gmail.cor

Double-check your email address before continuing.

Search Engine Visibility

☒ Discourage search engines from indexing this site

It is up to search engines to honor this request.

Install WordPress

Рисунок 2 — Настройка WordPress

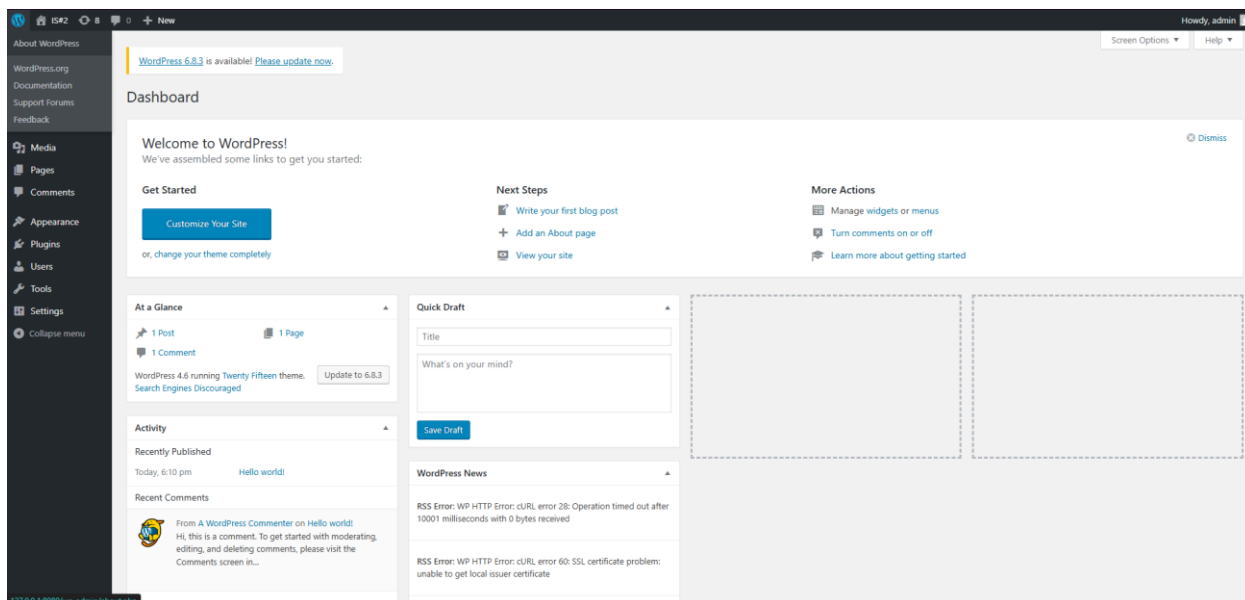


Рисунок 3 — Панель управления WordPress

6. Запуск атакующего скрипта на Python;

```
C:\Users\USER\AppData\Local\Programs\Python\Python313\python.exe exploit.py http://127.0.0.1:8080 payload/payload.txt admin
[+] Step 1: ask target to curl payload into /tmp/rce
[+] response status: 200
[+] Step 2: run downloaded payload
[+] response status: 200
[+] done - check /tmp on the web container
```

Рисунок 4 — Работа Python-скрипта

7. Проверка осуществления атаки на целевом устройстве. Должны создаваться 2 файла — **rce** (исполняемый файл) и **success** (результатирующий файл).

```
# ls
rce success
# cat success
Окак
# cat rce
#!/bin/sh
echo Окак > /tmp/success#
```

Рисунок 5 — Содержимые результирующего и исполняемого файлов

Анализ root cause (коренной причины)

Уязвимость касается системы управления контентом WordPress версии 4.6. Проблема заключалась в некорректной обработке пользовательских данных в широкоиспользуемой библиотеке *PHPMailer*, который встроен в WordPress для отправки и-мэйлов.

У WordPress есть своя функция для отправки электронной почты — *wp_mail()*, но в основе его работы лежит функция *mailSend()* из *PHPMailer*. Сама функция формировала параметр команды, используя поле *Sender*, и передаёт его в качестве аргумента *additional_parameters()* функции PHP *mail()*.

```
if (!empty($this->Sender)) {  
    $params = sprintf('-f%s', $this->Sender);  
}  
if ($this->Sender != '' and !ini_get('safe_mode')) {  
    $old_from = ini_get('sendmail_from');  
    ini_set('sendmail_from', $this->Sender);  
}  
...  
$result = $this->mailPassthru($to, $this->Subject, $body, $header,  
$params);
```

Рисунок 6 — кусок кода с уязвимостью

Поскольку библиотека не обеспечивала надлежащую очистку или нейтрализацию метасимволов оболочки (“\”, двойная кавычка \") в значении *Sender*, злоумышленник мог создать адрес электронной почты, включающий дополнительные флаги *sendMail* или аргументы команды. Это фактически позволяло внедрять аргументы в базовую команду *mail*.

В итоге удалённый (неаутентифицированный) злоумышленник мог передавать дополнительные параметры исполняемому файлу *sendMail* (через *mail()*), что потенциально приводило к выполнению кода на сервере.

Примененное исправление и его описание

Команда разработки WordPress выкатила обновление, где были следующие поправки,

```
if (!empty($this->Sender) and $this->validateAddress($this->Sender))  
{  
    $params = sprintf('-f%s', escapeshellarg($this->Sender));  
}
```

Рисунок 7 — кусок обновленного кода с валидаторами

где:

- *validateAddress()* отвечает за валидацию адреса получателя;
- *escapeshellarg()* отвечает за оборачивания/экранирования значения отправителя так, чтобы оно не может быть разделено на аргументы оболочки.

Для того, чтобы устранить уязвимость в нашем случае, нам достаточно всего лишь обновить WordPress до безопасной версии, точнее 4.7.1. Проще всего делать обновление в самом файле docker-compose.yml:

```
services:
  web:
    image: wordpress:4.7.1-apache
```

Рисунок 8 — кусок обновленного файла конфигурации Docker

Доказательство устранения уязвимости

Воспроизведём те же шаги из раздела 2, но при запуске скрипта в ответ приходит ошибка:

```
C:\Users\USER\AppData\Local\Programs\Python\Python313\python.exe exploit.py http://127.0.0.1:8080 payload/payload.txt admin
[+] Step 1: ask target to curl payload into /tmp/rce
[+] response status: 500
[+] Step 2: run downloaded payload
[+] response status: 500
[+] done - check /tmp on the web container
```

Рисунок 9 — Работа Python-скрипта, но в ответ код ошибки 500

И если проверить содержимые целевого устройства,

```
# cd tmp
# ls
pear
```

Рисунок 10 — Содержимые директории /tmp

То можно заметить, что в директории не удалось создать исполняемый файл.